RouteWise

Detaylı Sistem Analizi ve Akış Şeması

Lojistik Dağıtım Süreçlerinde Dinamik Rota ve Araç Optimizasyonu Sağlayan Akıllı Yazılım Sistemi

Kapsam: Bandırma içi kargo dağıtımı — tek dağıtım merkezi

Problem sınıfı: CVRP + maksimum rota süresi + dinamik yeniden rotalama

Yığın: Spring Boot · React · PostgreSQL/PostGIS · Python

Repo: github.com/AllenVB/RouteWise

Sürüm: Taslak v1 · 10.10.2026 — *revizyon için*

Bu doküman revizyon içindir. Veri modeli ve bazı iş kuralları bu çalışmada ilk kez tasarlanmıştır; **varsayım** olarak işaretlenen noktalar danışman görüşüne açıktır ve değiştirilebilir.

Tanım ve Yapı

02 Problem tanımı, kapsam ve kısıtlar
03 Teknoloji yığını — kullanılan her şey
04 Sistem mimarisi ve bileşenler
05 Docker dağıtım mimarisi

Veri Modeli

06 ER diyagramı — genel görünüm
07 Tablo detayları I — kullanıcı, rol, adres
08 Tablo detayları II — kargo, rota, optimizasyon
09 İlişki kardinaliteleri ve bütünlük kuralları

Süreçler

10 Roller, yetkiler ve durum makineleri
11 Veri hattı — OSM'den yol grafına
12 Optimizasyon akışı
13 Dinamik yeniden rotalama senaryoları

Algoritma

14 RouteWise algoritması
15 Amaç fonksiyonu — maliyet modeli
16 Algoritma karşılaştırması — iki seviye

Çıktı

17 Analiz, Excel raporları ve tasarruf yüzdesi
18 Geliştirme yol haritası

Çözülen problem

CVRP + maksimum rota süresi — Capacitated Vehicle Routing Problem, tek depo, çalışma günü süre sınırı ile; üstüne dağıtım sırasında **dinamik yeniden rotalama**.

Bandırma'daki tek kargo şubesinden çıkan araçlar, kapasiteleri kadar paketi gün içinde teslim edip şubeye döner. Sistem bu dağıtımı yalnızca mesafe değil, **yakıt, süre, sürücü maliyeti ve teslimat önceliğini birlikte** değerlendirerek planlar.

Kısıtlar ve değişkenler

Boyut	Tip	Modelleme
Araç kapasitesi	**SERT**	Σ paket ağırlığı ≤ kapasite · aşılamaz
Çalışma günü süresi	**SERT**	toplam rota süresi ≤ gün sınırı
Teslimat önceliği	**YUMUŞAK**	amaç fonksiyonunda ceza katsayısı
Toplam mesafe	amaç	yol grafı + Dijkstra
Yakıt tüketimi	amaç	araç bazlı L/100km × km
Sürücü maliyeti	amaç	süre × saatlik ücret
Dağıtım merkezi	yapısal	**tek** depo

Kapsam dışı: müşteri bazlı zaman penceresi (VRPTW). "Tüm teslimatlar gün içinde bitsin" rota başına **tek bir süre üst sınırıdır**; "bu paket 09:00–12:00 arası teslim edilmeli" değildir. İkincisi her teslimat için ayrı olurluluk kontrolü, varış saati yayılımı ve bekleme süresi hesabı gerektirir ve problem sınıfını değiştirir.

Projenin özgün yanı

Sistem **yalnızca en kısa rotayı** bulmaz. Maliyet, rota hesaplandıktan sonra eklenen bir rapor alanı değil, **amaç fonksiyonunun parçasıdır** — algoritmanın kararlarını doğrudan bu değerler yönlendirir. (bkz. sayfa 15)

Başarı kriterleri — proje önerisinden

- Teslimat noktalarına göre **otomatik ve optimize** rota oluşturulabilmesi
- Teslimat bilgisi değiştiğinde rotanın **yeniden oluşturulabilmesi**
- Araç kapasitesi ve önceliklerin planlamaya dahil edilebilmesi
- Toplam mesafe ve **tahmini yakıt tüketiminin** hesaplanabilmesi
- Optimize rota ile **standart rota farkının ölçülebilmesi**
- Dağıtım verilerinin saklanabilmesi
- Haftalık / aylık / yıllık **tasarruf analizlerinin raporlanabilmesi**
- Farklı araç ve dağıtım senaryolarında çalıştırılabilmesi
- Geleneksel planlamaya kıyasla **ölçülebilir verimlilik artışı**

Kapsam daraltma kararı

Çalışmanın erken aşamasında **çok depo + zaman penceresi (MDVRPTW)** düşünülmüştü. Sistem tarifi netleştikçe kapsam **tek depo + gün sınırına** daraltıldı. Bu daraltmanın üç somut getirisi oldu:
• Klasik, iyi dokümante edilmiş algoritmalar (savings, 2-opt) kullanılabilir hale geldi
• Bandırma ölçeğinde **kendi yol grafımızı** kurmak ucuzladı → harici rota API'si gerekmiyor
• `osm2pgrouting`'in bellek kısıtı önemsizleşti

03 Teknoloji Yığını

Backend

Bileşen	Sürüm / Not
Java	21 (LTS)
Spring Boot	3.4+
Spring Web	REST API
Spring Security	JWT kimlik doğrulama
Spring Data JPA	ORM
Hibernate Spatial	PostGIS tipleri
Liquibase	Şema migration + seed
Spring Mail	E-posta bildirimi
Gradle	Derleme

Veritabanı

Bileşen	Sürüm / Not
PostgreSQL	17
PostGIS	3.6 — coğrafi tipler/sorgular
pgRouting	3.8 — yol grafı, referans Dijkstra

Veri hazırlığı

Araç	İş
OpenStreetMap	Ham yol verisi (ODbL)
Geofabrik	`turkey-latest.osm.pbf` ~619 MB
osmium-tool	Bandırma poligonuyla kırpma
osm2pgrouting	Grafı PostGIS'e yükleme

Python servisi

Bileşen	Sürüm / Not
Python	3.12
FastAPI	REST servis
uvicorn	ASGI sunucu
pydantic	İstek/yanıt doğrulama
numpy	Matris işlemleri
pandas	Agregasyon, zaman serisi
scipy	**Anlamlılık testleri**
matplotlib	Yayın kalitesi figür (SVG/PDF)
plotly	İnteraktif grafik
openpyxl	Excel dışa aktarma
pytest	Test

Frontend

Bileşen	Sürüm / Not
React	19
TypeScript	strict mod
Vite	Derleme / dev sunucu
MapLibre GL	Harita, rota çizimi
TanStack Query	Sunucu durumu
react-hook-form + zod	Form + doğrulama
Recharts	İnteraktif grafik

Altyapı

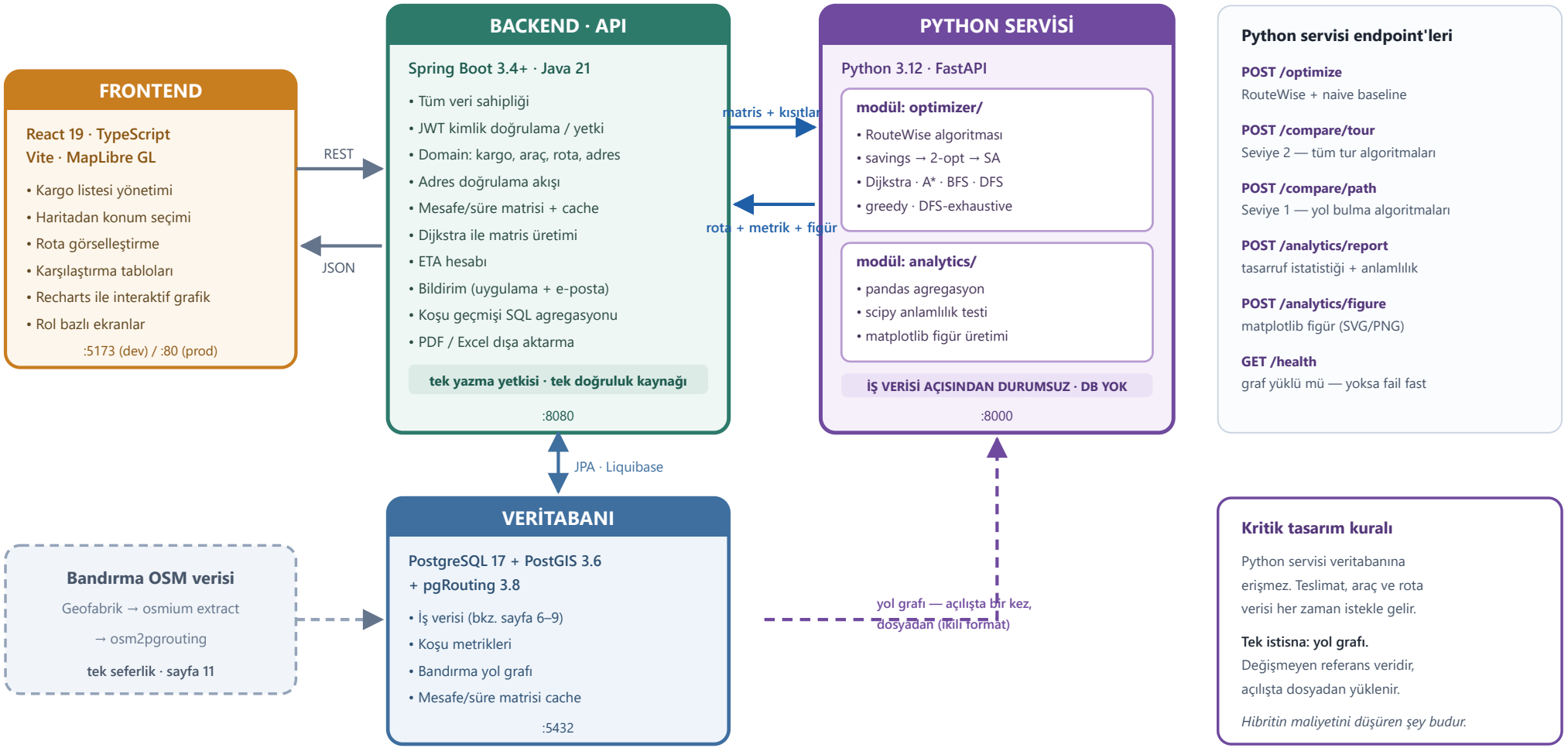
Bileşen	Not
Docker	Tüm servisler container'da
Docker Compose	Orkestrasyon
nginx	Frontend prod sunumu
Testcontainers	Gerçek DB ile entegrasyon testi

Algoritmalar — kendi implementasyonumuz

Algoritma	Rolü
Clarke-Wright savings	Başlangıç çözümü
2-opt	Rota içi yerel arama
relocate / exchange / Or-opt	Rotalar arası yerel arama
Simulated annealing	Metasezgisel
Dijkstra	Matris + referans optimum
A*	Karşılaştırma
BFS / DFS	Karşılaştırma
Nearest Neighbor	Karşılaştırma taban çizgisi

Hazır çözücü kullanılmıyor. OR-Tools veya benzeri bir VRP kütüphanesi yerine algoritma projeye özgü geliştirilir; özgünlük iddiasının kodda karşılığı budur.

Harici rota API'si yok. Google Maps / OpenRouteService kullanılmaz — mesafe ve süre matrisi kendi yol grafımızdan hesaplanır. Kota, API anahtarı ve internet bağımlılığı yoktur.



Python servisi endpoint'leri

POST /optimize
RouteWise + naive baseline

POST /compare/tour
Seviye 2 — tüm tur algoritmaları

POST /compare/path
Seviye 1 — yol bulma algoritmaları

POST /analytics/report
tasarruf istatistiği + anlamlılık

POST /analytics/figure
matplotlib figür (SVG/PNG)

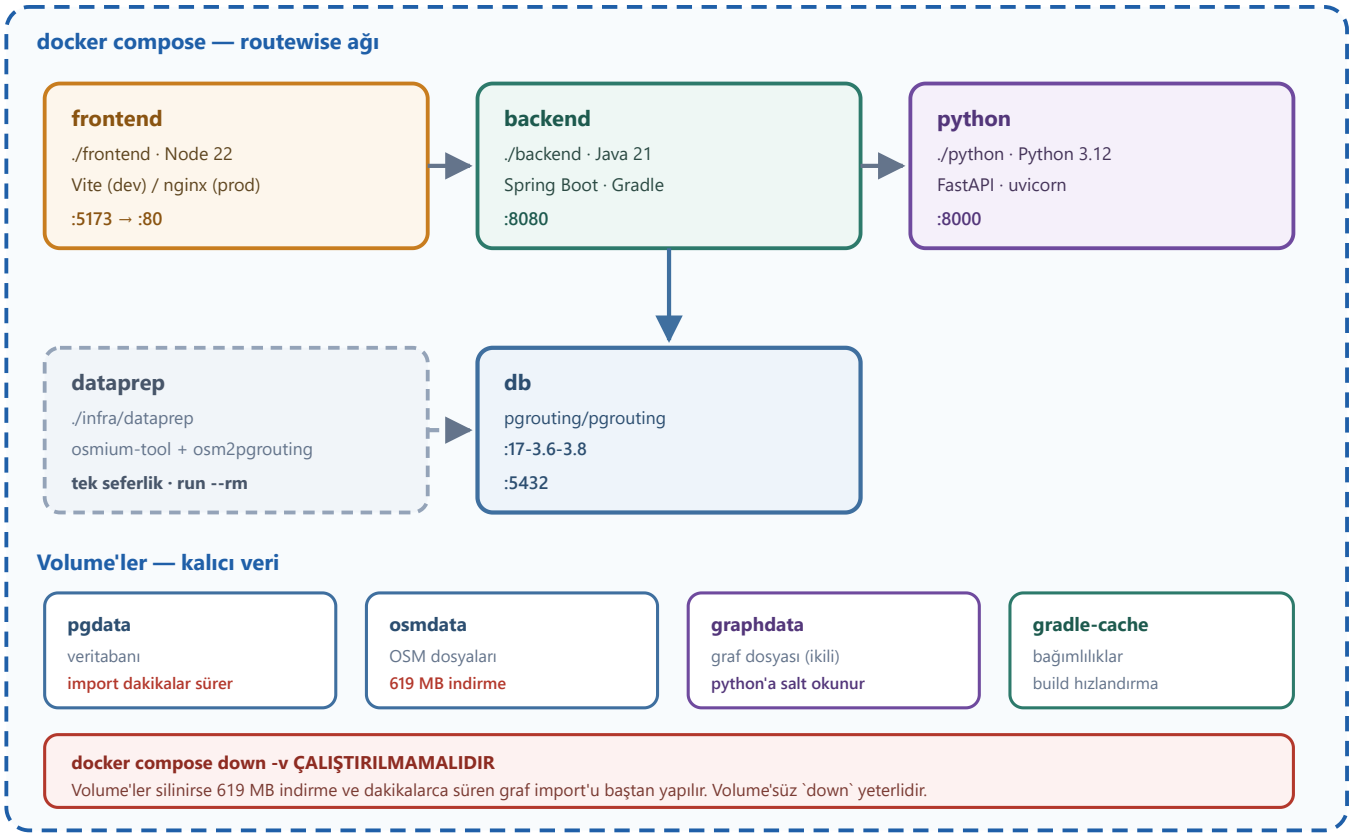
GET /health
graf yüklü mü — yoksa fail fast

Kritik tasarım kuralı

Python servisi veritabanına erişmez. Teslimat, araç ve rota verisi her zaman istekle gelir.

Tek istisna: yol grafı.
Değişmeyen referans veridir, açılışta dosyadan yüklenir.

Hibritin maliyetini düşüren şey budur.



Başlatma sırası ve healthcheck

db	— · pg_isready
backend	db healthy · /actuator/health
python	— · /health graf yoksa FAIL FAST
frontend	backend (gevşek) · HTTP 200

Python servisi graf dosyasını bulamazsa sessizce başlamaz — hatayı erken ve açıkça verir.

Veri hazırlığı — tek seferlik, 4 adım

```
compose run --rm dataprep download
compose run --rm dataprep clip
compose run --rm dataprep load
compose run --rm dataprep export
```

Adımlar ayrıdır: biri patlarsa baştan başlanmaz.

Windows'a özgü zorunlu ayarlar

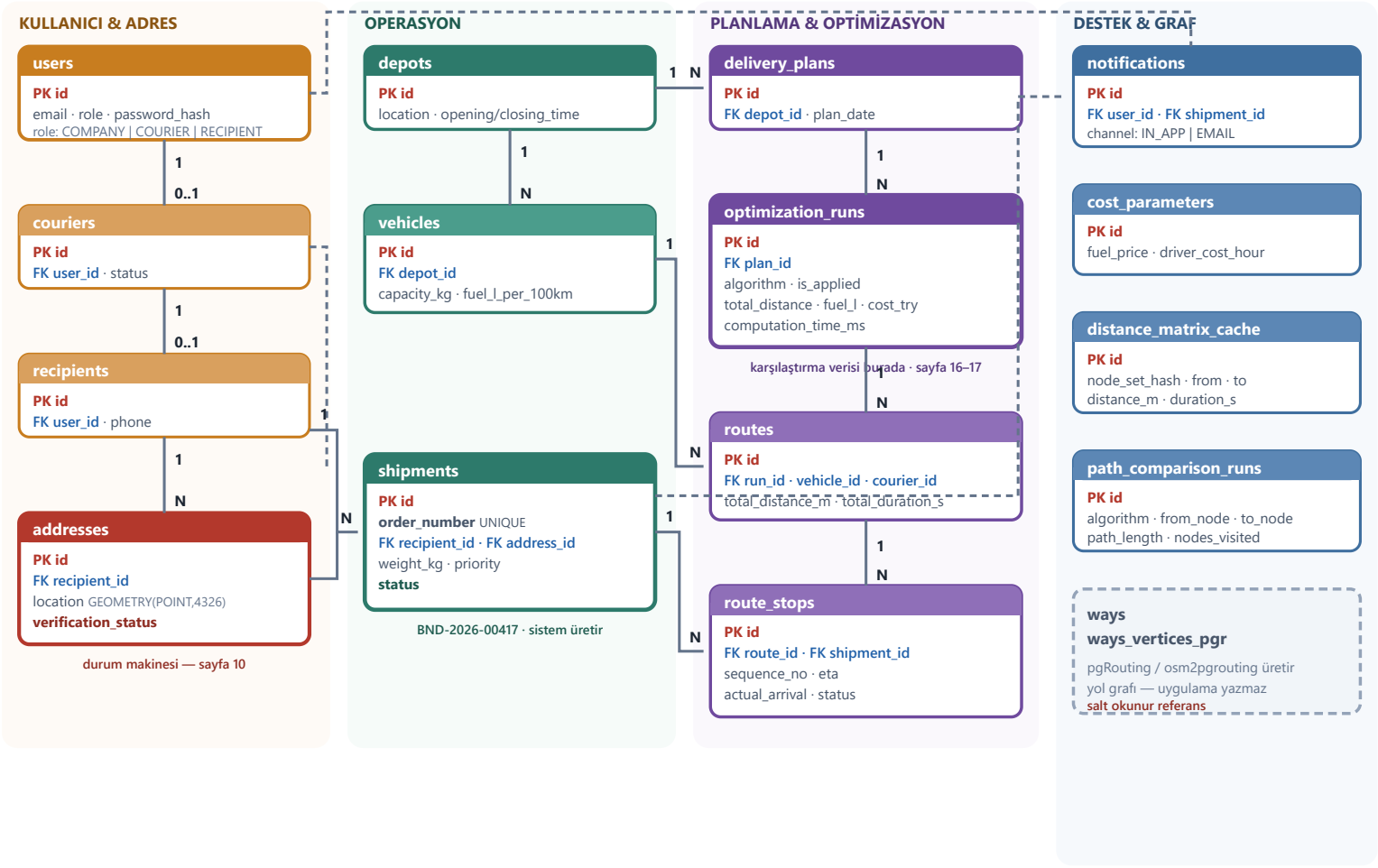
Vite hot reload tetiklenmez
bind-mount dosya olayları iletilmez → watch.usePolling: true · host: '0.0.0.0'

Spring Boot her değişimde imaj derlerse
iterasyon ölür → bootRun + devtools, kaynak bind-mount, .gradle named volume

Her restart'ta graf yeniden yüklenir
→ kompakt ikili format, JSON değil

Yapılandırma ve ortam

- Sırlar ve ortam ayarları **`.env`** dosyasında; **`.env.example`** repoda versiyonlanır, **`.env`** .gitignore'da
- Servisler birbirine **servis adıyla** erişir (db, backend, python) — IP veya localhost değil
- **postgis/postgis kullanılmıyor**: pgRouting içermiyor. Alternatif: nickblah/pgrouting (yama sürümüne kadar sabitlenebilir etiket)
- Geliştirme ve teslim **aynı compose** üzerinden — danışman/jüri tek komutla sistemi ayağa kaldırabilmeli



Gösterim

PK birincil anahtar

FK yabancı anahtar

1 — N bire-çok ilişki

1 — 0..1 isteğe bağlı bire-bir

Kesikli çizgi: zayıf / opsiyonel bağ

Kalın çerçeve: iş kuralı yoğun tablo

Tüm alanlar: sayfa 7–8

VARSAYIM — revize edilebilir

1. Alıcı **hesap sahibidir**; kargo bir alıcı kullanıcısına bağlanır. Hesapsız alıcı senaryosu modellenmedi.
2. Kurye–araç ataması **rota üzerinden** yapılır (günlük). Kalıcı araç sahipliği yok.
3. Bir kargo farklı günlerde yeniden planlanabilir → **shipments 1:N route_stops**.
4. Depo tablosu tek kayıt tutar ama çoklu depoya açıktır.

Özet

13 uygulama tablosu + 2 pgRouting tablosu

Coğrafi alan: addresses.location, depots.location

users

Alan	Tip	Kısıt
id	BIGSERIAL	PK
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL · BCrypt
full_name	VARCHAR(150)	NOT NULL
phone	VARCHAR(20)	
role	VARCHAR(20)	NOT NULL
is_active	BOOLEAN	DEFAULT true
created_at	TIMESTAMPTZ	NOT NULL
updated_at	TIMESTAMPTZ	

role: COMPANY · COURIER · RECIPIENT. Rol bazlı ek alanlar ayrı profil tablolarında tutulur — tek tabloda null alan yığılmaz.

couriers

Alan	Tip	Kısıt
id	BIGSERIAL	PK
user_id	BIGINT	FK→users · **UNIQUE**
employee_code	VARCHAR(30)	UNIQUE
status	VARCHAR(20)	NOT NULL
created_at	TIMESTAMPTZ	

status: AVAILABLE · ON_ROUTE · OFF_DUTY. `user_id` UNIQUE olduğu için ilişki **1:1**'dir.

recipients

Alan	Tip	Kısıt
id	BIGSERIAL	PK
user_id	BIGINT	FK→users · **UNIQUE**
default_address_id	BIGINT	FK→addresses · NULL
created_at	TIMESTAMPTZ	

addresses iş kuralı yoğun

Alan	Tip	Kısıt
id	BIGSERIAL	PK
recipient_id	BIGINT	FK→recipients, NOT NULL
address_text	VARCHAR(500)	NOT NULL
district	VARCHAR(100)	
neighbourhood	VARCHAR(100)	
location	GEOMETRY (POINT,4326)	NOT NULL · **GIST index**
snapped_node_id	BIGINT	→ ways_vertices_pgr
snap_distance_m	NUMERIC(8,2)	yola uzaklık
verification_status	VARCHAR(20)	NOT NULL
verified_by	BIGINT	FK→users · NULL
verified_at	TIMESTAMPTZ	NULL
rejection_reason	VARCHAR(300)	NULL
change_count_today	SMALLINT	DEFAULT 0 · limit
created_at / updated_at	TIMESTAMPTZ	

verification_status: DRAFT · PENDING · VERIFIED · REJECTED. **Yalnızca VERIFIED adresler optimizasyona girer.** Alıcı adresi değiştirdiğinde durum otomatik PENDING'e düşer — aksi hâlde firma doğruladığını sanırken başka adrese rota çizilir.

snapped_node_id çifte iş görür: hem yol grafına bağlanma noktasıdır (rota hesabı için zorunlu), hem de yanlış işaretleme kontrolüdür — `snap_distance_m` eşiği aşarsa nokta reddedilir.

depots

Alan	Tip	Kısıt
id	BIGSERIAL	PK
name	VARCHAR(120)	NOT NULL
address_text	VARCHAR(500)	
location	GEOMETRY (POINT,4326)	NOT NULL · GIST
snapped_node_id	BIGINT	→ ways_vertices_pgr
opening_time	TIME	çıkış saati
closing_time	TIME	dönüş saati
max_route_duration_min	INTEGER	NOT NULL

max_route_duration_min projenin ikinci sert kısıtıdır — "tüm teslimatlar gün içinde bitsin" kuralının veri karşılığı. Rota süresi bu değeri aşamaz.

vehicles

Alan	Tip	Kısıt
id	BIGSERIAL	PK
depot_id	BIGINT	FK→depots, NOT NULL
plate	VARCHAR(20)	UNIQUE, NOT NULL
capacity_kg	NUMERIC(8,2)	NOT NULL · sert kısıt
capacity_volume_m3	NUMERIC(8,2)	
fuel_l_per_100km	NUMERIC(5,2)	NOT NULL · maliyet
fuel_type	VARCHAR(20)	DIESEL · GASOLINE
fixed_cost_per_day	NUMERIC(10,2)	opsiyonel bileşen
is_active	BOOLEAN	DEFAULT true

Filo şimdilik sabittir — Liquibase seed verisiyle yüklenir, yönetim ekranı yazılmaz. Ancak koda gömülmez: tablo baştan vardır, böylece "farklı araç senaryolarında çalıştırılabilmesi" kriteri sonradan karşılanabilir.

shipments kargo

Alan	Tip	Kısıt
id	BIGSERIAL	PK
order_number	VARCHAR(20)	UNIQUE, NOT NULL
recipient_id	BIGINT	FK→recipients
address_id	BIGINT	FK→addresses
weight_kg	NUMERIC(8,2)	NOT NULL · kapasite
volume_m3	NUMERIC(8,2)	
priority	SMALLINT	DEFAULT 0 · ceza
service_time_min	INTEGER	teslim süresi
status	VARCHAR(20)	NOT NULL
postponed_to	DATE	NULL · erteleme
created_at / updated_at	TIMESTAMPTZ	

order_number sistem tarafından üretilir (`BND-2026-00417`) — elle giriş yoktur, yanlış yazma riski sıfırdır.
status: BEKLEMEDE · PLANLANDI · YOLDA · TESLİM_EDİLDİ · ERTELENDİ.

delivery_plans günlük plan

Alan	Tip	Kısıt
id	BIGSERIAL	PK
depot_id	BIGINT	FK→depots
plan_date	DATE	NOT NULL
status	VARCHAR(20)	TASLAK·SEVK·TAMAM
created_by	BIGINT	FK→users
created_at	TIMESTAMPTZ	
UNIQUE (depot_id, plan_date) — gün başına tek plan		

optimization_runs her algoritma koşusu

Alan	Tip	Kısıt
id	BIGSERIAL	PK
plan_id	BIGINT	FK→delivery_plans
algorithm	VARCHAR(40)	NOT NULL
is_applied	BOOLEAN	uygulanan rota mı
is_baseline	BOOLEAN	naive referans mı
total_distance_m	NUMERIC(12,2)	
total_duration_s	INTEGER	
total_fuel_l	NUMERIC(10,3)	
total_cost_try	NUMERIC(12,2)	tasarruf hesabı
vehicle_count	SMALLINT	
computation_time_ms	INTEGER	hız karşılaştırması
random_seed	BIGINT	SA tekrarlanabilirlik
created_at	TIMESTAMPTZ	

Projenin ölçüm omurgası bu tablodur. Her koşuda RouteWise *ve* naive baseline kaydedilir; tasarruf yüzdesi ikisinin `total_cost_try` farkından türetilir. Karşılaştırma koşusunda aynı `plan_id` altında her algoritma için bir satır oluşur.

algorithm: NAIVE · GREEDY_NN · SAVINGS · ROUTEWISE · DFS_EXHAUSTIVE.
random_seed olmadan SA aynı girdiye farklı sonuç verir ve sonuçlar tekrar üretilemez — tez için zorunlu alan.

routes araç başına rota

Alan	Tip	Kısıt
id	BIGSERIAL	PK
run_id	BIGINT	FK→optimization_runs
vehicle_id	BIGINT	FK→vehicles
courier_id	BIGINT	FK→couriers · NULL
total_distance_m	NUMERIC(12,2)	
total_duration_s	INTEGER	≤ max_route_duration
total_load_kg	NUMERIC(8,2)	≤ capacity_kg
status	VARCHAR(20)	
started_at / completed_at	TIMESTAMPTZ	NULL

route_stops duraklar

Alan	Tip	Kısıt
id	BIGSERIAL	PK
route_id	BIGINT	FK→routes
shipment_id	BIGINT	FK→shipments
sequence_no	INTEGER	NOT NULL · ziyaret sırası
distance_from_prev_m	NUMERIC(10,2)	
duration_from_prev_s	INTEGER	
eta	TIMESTAMPTZ	tahmini varış
actual_arrival	TIMESTAMPTZ	NULL
status	VARCHAR(20)	
UNIQUE (route_id, sequence_no)		

eta alanı alıcıya gönderilen bildirimin kaynağıdır (sayfa 12). `actual_arrival` ile farkı, tahmin doğruluğunun ölçülmesini sağlar — raporda ek bir metrik olarak kullanılabilir.

09 Destek Tabloları, Kardinaliteler ve Bütünlük Kuralları

notifications

Alan	Tip	Kısıt
id	BIGSERIAL	PK
user_id	BIGINT	FK→users
shipment_id	BIGINT	FK→shipments · NULL
type	VARCHAR(30)	ETA·ERTELEME·DOĞRULAMA
channel	VARCHAR(20)	IN_APP · EMAIL
payload	JSONB	mesaj içeriği
sent_at	TIMESTAMPTZ	
read_at	TIMESTAMPTZ	NULL

SMS kapsam dışıdır; `channel` alanı ileride genişletilebilir şekilde tasarlanmıştır.

cost_parameters

Alan	Tip	Kısıt
id	BIGSERIAL	PK
fuel_price_per_l	NUMERIC(8,2)	NOT NULL
driver_cost_per_hour	NUMERIC(8,2)	NOT NULL
priority_penalty	NUMERIC(8,2)	ceza katsayısı
effective_from	DATE	NOT NULL

Parametreler **tarihli** tutulur: yakıt fiyatı değiştiğinde geçmiş koşuların maliyeti bozulmaz. Her optimizasyon çağrısında o günün parametreleri Python servisine gönderilir — kod değişmeden sonuçlar değişir.

distance_matrix_cache

Alan	Tip	Kısıt
id	BIGSERIAL	PK
node_set_hash	CHAR(64)	SHA-256 · INDEX
from_node_id	BIGINT	NOT NULL
to_node_id	BIGINT	NOT NULL
distance_m	NUMERIC(10,2)	Dijkstra sonucu
duration_s	INTEGER	maxspeed'den
computed_at	TIMESTAMPTZ	
UNIQUE (node_set_hash, from_node_id, to_node_id)		

Bu cache olmadan sistem kullanılamaz. Metasezgisel algoritma matrisi koşu başına bir kez ister, ama algoritma ayarı sırasında yüzlerce koşu yapılır. Aynı durak kümesi → aynı hash → tekrar hesap yok.

path_comparison_runs

Alan	Tip	Kısıt
id	BIGSERIAL	PK
algorithm	VARCHAR(20)	DIJKSTRA·ASTAR·BFS·DFS
from_node_id / to_node_id	BIGINT	NOT NULL
path_length_m	NUMERIC(10,2)	
computation_time_ms	NUMERIC(10,3)	
nodes_visited	INTEGER	
deviation_pct	NUMERIC(6,2)	Dijkstra'ya göre
created_at	TIMESTAMPTZ	

Seviye 1 (yol bulma) deneylerinin kaydı. Seviye 2 (tur sırası) kayıtları `optimization_runs` tablosundadır. İki seviye ayrı tutulur çünkü ölçülen metrikler farklıdır — bkz. sayfa 16.

Tüm ilişkiler ve kardinaliteler

İlişki	Kard.	Açıklama
users → couriers	**1 : 0..1**	profil
users → recipients	**1 : 0..1**	profil
users → notifications	**1 : N**	
recipients → addresses	**1 : N**	çoklu adres
recipients → shipments	**1 : N**	
addresses → shipments	**1 : N**	aynı adrese çok kargo
depots → vehicles	**1 : N**	
depots → delivery_plans	**1 : N**	gün başına bir plan
delivery_plans → optimization_runs	**1 : N**	her algoritma bir koşu
optimization_runs → routes	**1 : N**	her araç bir rota
routes → route_stops	**1 : N**	sıralı duraklar
shipments → route_stops	**1 : N**	farklı koşu/günde tekrar
vehicles → routes	**1 : N**	
couriers → routes	**1 : N**	günlük atama
shipments → notifications	**1 : N**	

Bütünlük kuralları

1. Yalnızca `verification_status = VERIFIED` adresli kargo plana dahil edilir.
2. Bir plan altında en fazla **bir** koşu `is_applied = true` olabilir.
3. `routes.total_load_kg ≤ vehicles.capacity_kg` — uygulama ve algoritma seviyesinde zorlanır.
4. `routes.total_duration_s ≤ depots.max_route_duration_min × 60`
5. Alıcı adresi değiştirince `verification_status → PENDING` (tetikleyici / servis katmanı).
6. Kargo `YOLDA` iken adres **değiştirilemez**.
7. Silme davranışı: profil ve rota zincirinde `ON DELETE CASCADE`; kargo ve koşu kayıtlarında `ON DELETE RESTRICT` — geçmiş ölçüm verisi korunur.

Yetki matrisi

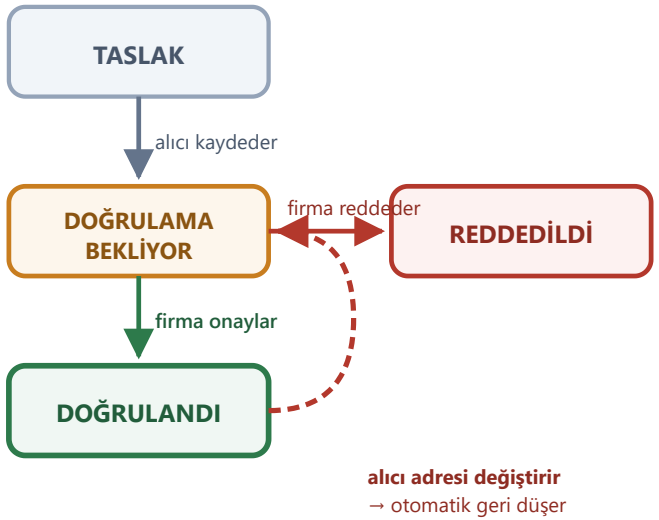
Yetki	Firma	Kurye	Alıcı
Kargo kaydı oluşturma	✓	—	—
Sipariş no üretimi	otomatik	—	—
Adres doğrulama / reddetme	✓	—	—
Rota oluşturma, optimizasyon	✓	—	—
Rotayı görme	tümü	kendi	kendi teslimatı
Teslimat durumu güncelleme	✓	kendi durakları	—
Kendi adresini değiştirme	✓	—	✓
Teslimatı erteleme	✓	—	✓
Araç / filo yönetimi	seed	—	—
Karşılaştırma koşusu başlatma	✓	—	—
Tasarruf raporları, Excel/PDF	✓	—	—

Kimlik doğrulama **JWT** ile; her endpoint rol bazlı korunur. Test stratejisinde her endpoint için **rol sızıntısı testi** zorunludur — alıcı başkasının teslimatını görememelidir.

Alıcı adresi doğrudan değiştirebilir — güçlü bir özelliktir ama tasarlanmazsa sistemi kırar. Zorunlu önlemler:

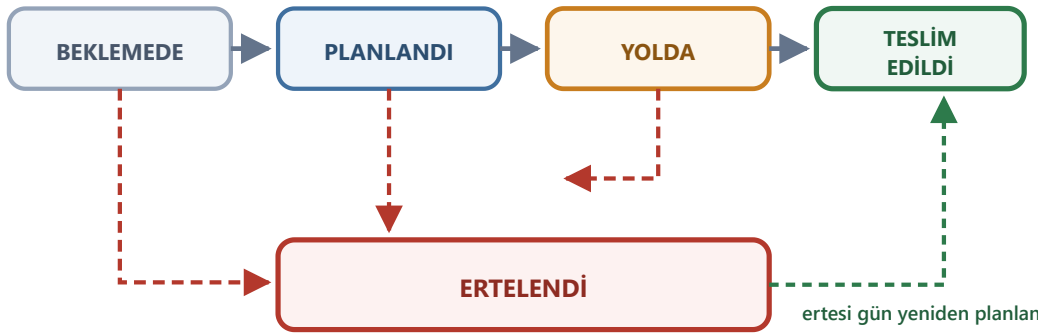
- Yeni adres rotayı süre sınırının üstüne çıkarırsa teslimat **otomatik ERTELENDİ'ye** düşer, alıcı bilgilendirilir
- Teslimat **YOLDA** olunca adres **kilitlenir**
- Maliyet artışı eşiği aşarsa sistem erteleme önerir
- Teslimat başına **günlük değişiklik limiti**
- Kuryeye **açık bildirim**; eski durak listesinden kaldırılır

Adres doğrulama durum makinesi



Yalnızca DOĞRULANDI adresler optimizasyona girer.
Bu kural olmadan firma doğruladığını sanırken başka adrese rota çizilir.

Teslimat durum makinesi



ERTELENDİ'ye geçiş sebepleri

- Alıcı "başka gün gelsin" der
- Teslimat denemesi başarısız
- Adres değişikliği rotayı süre sınırının üstüne çıkarır → **otomatik** durak kümeden çıkar, rota yeniden hesaplanır

11 Veri Hattı — OSM'den Yol Grafına

Tek seferlik kurulum · harici API yok

KURULUM HATTI — bir kez çalışır

1 download
Geofabrik
turkey-latest.osm.pbf
619 MB
08.10.2026 verisi

→

2 clip
osmium-tool
osmium extract --polygon
bandirma.poly
→ birkaç on MB

→

3 load
osm2pgrouting 3.0
→ ways
→ ways_vertices_pgr
topolojiyi kendi kurar

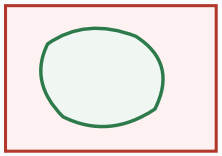
→

4 export
graf → ikili dosya
graphdata volume'üne
Python açılışta okur
JSON değil — restart hızı

Neden kendimiz kırpıyoruz

Geofabrik Türkiye için alt bölge yayınlamıyor:
"No sub regions are defined for this region."
Tek dosya var.

bbox değil, poligon



dikdörtgen vs ilçe sınırı

Bbox dikdörtgeni komşu ilçelerden gereksiz yol ağı taşır: graf şişer, Dijkstra yavaşlar.

Aynı poligon geofence kontrolünde de kullanılır — bir iş, iki fayda.

ÇALIŞMA ZAMANI — matris hesabı

Durak listesi
depo + teslimat noktaları

→

Kendi Dijkstra'mız
graf üzerinde, her çift için

→

Mesafe + süre matrisi
N × N
süre = uzunluk / maxspeed

↔

distance_matrix_cache
node_set_hash ile aranır
aynı küme → tekrar hesap yok

→

Python
optimizer

Harici rota API'si neden gerekmiyor

Hazır API ile (OpenRouteService / Google)	Kendi grafımızla
Günlük istek kotası	sınırsız
Matris başına ≈59 konum sınırı	sınır yok
API anahtarı yönetimi	gerekmez
Demo gününde internet riski	tamamen yerel
Ham graf alınamaz	graf elimizde

Graf olmadan Seviye 1 (Dijkstra/A*/BFS/DFS) karşılaştırması zaten yapılamazdı.

Doğruluk referansı

pgRouting'in kendi `pgr_dijkstra` fonksiyonu, bizim Dijkstra implementasyonumuzun doğruluk referansıdır: **aynı girdi → aynı mesafe.**

Test stratejisinin zorunlu maddesidir (sayfa 18).

osm2pgrouting — bilinen kısıt

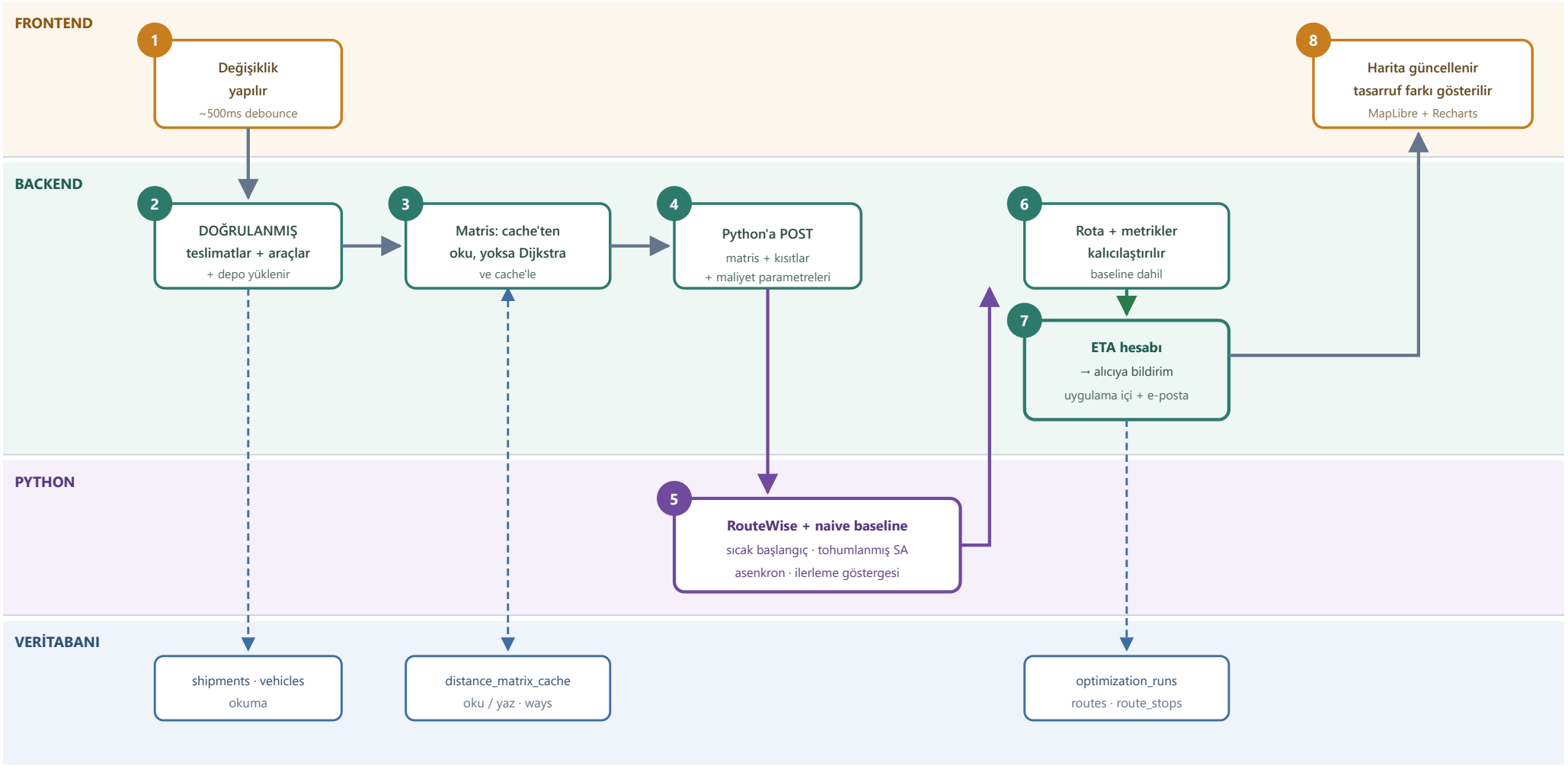
Araç veriyi **tamamen belleğe yükler**; bu, büyük ağlarda ciddi bir sınırdır.

Bandırma ölçeğinde önemsizdir — kapsam daraltmanın üçüncü somut getirisi.

Araç bakımlıdır: Debian'da 3.0.0 sürümü Mayıs 2026'da testing'e geçmiştir.

TETİKLEYİCİLER — her biri anında yeniden hesaplama başlatır

- Paket eklendi / çıkarıldı
- Alıcı adresi değiştirdi
- Alıcı "başka gün gelsin" dedi
- Firma adresi doğruladı
- Teslimat tamamlandı (dağıtım sırasında)



İlke

Dağıtıma çıkmadan önce tam optimizasyon koşar ve en tasarruflu rota seçilir. Dağıtım sırasında bir olay olursa: **teslim edilmiş duraklar sabitlenir**, kalan duraklar yeniden optimize edilir ve kuryeye bildirilir.

Canlı GPS gerekmiyor

Yeniden optimizasyon **son tamamlanan duraktan** başlar, canlı konumdan değil.

Kurye her teslimatı tamamlandı işaretlediği için sistem nerede olduğunu zaten bilir.

SENARYO A — Alıcı "başka gün gelsin" der



4 durak · 18,4 km

3 durak · 14,1 km · −23%

Kargo **ERTELENDİ** durumuna geçer, `postponed_to` alanı dolar.

Rota yeniden hesaplanır, kuryeye bildirilir. Kargo ertesi gün yeniden planlanır.

Risk düşüktür: durak yalnızca kümeden çıkar, yeni kısıt gelmez.

SENARYO B — Alıcı adresini değiştirir



Adres **DOĞRULAMA_BEKLİYOR**'a düşer; firma onaylayana kadar rotaya girmez.

Onaylanınca kalan duraklar yeniden optimize edilir.

Risk yüksektir: yeni konum rotayı olurluluk sınırının dışına çıkarabilir.

Bu yüzden aşağıdaki beş önlem zorunludur.

Zorunlu güvenlik önlemleri — tasarlanmazsa sistem kırılır

Risk	Önlem
Yeni adres rotayı maksimum süre sınırının üstüne çıkarır	Teslimat **otomatik ERTELENDİ**'ye düşer, alıcı bilgilendirilir
Kurye o durağa giderken adres değişir	Teslimat **YOLDA** olunca adres kilitlenir, değişiklik kabul edilmez
Kurye o bölgeden çoktan geçmiş, yeni nokta çok geride	Maliyet artışı **eşiği aşarsa** sistem erteleme önerir
Tekrarlayan değişiklikle rota titreşimi	Teslimat başına **günlük değişiklik limiti** (change_count_today)
Kurye sessizce eski adrese gider	Kuryeye **açık bildirim**; eski durak listesinden kaldırılır

14 RouteWise Algoritması

Hazır çözücü yok · projeye özgü implementasyon

GİRDİ

- Mesafe matrisi (N×N)
- Süre matrisi (N×N)
- Teslimatlar: ağırlık, öncelik
- Araçlar: kapasite, tüketim
- Depo: max_route_duration
- Maliyet parametreleri
- **random_seed**

1 Clarke-Wright savings

Başlangıç çözümü.

Her teslimat tek başına bir rotayken başlanır; birleştirmenin kazancı

$s(i,j) = d(0,i) + d(0,j) − d(i,j)$

en yüksek olan çiftler sırayla birleştirilir.

Kapasite ve süre farkında: olursuz birleştirme atlanır.

2 Yerel arama

2-opt	rota içi — iki kenarı değiştirir
relocate	durağı başka rotaya taşır
exchange	iki durağı takas eder
Or-opt	2–3 duraklık dizi taşır

her hamlede olurluluk kontrol edilir

3 Simulated annealing

Kötüleştiren hamleler de olasılıkla kabul edilir → yerel optimumdan kaçış.

$P = \exp(-\Delta / T)$, T azalır

Tohumlanmış (seeded): tohumsuz aynı girdi farklı rota verir, harita sebepsiz zıplar, sonuç tekrar üretilemez.

Olurluluk kapısı — her hamlede

Her yerel arama hamlesi ve her SA adımı, kabul edilmeden önce iki sert kısıta karşı sınanır:

Σ ağırlık ≤ araç kapasitesi	vehicles.capacity_kg
toplam süre ≤ gün sınırı	depots.max_route_duration_min

Bu kontroller olmadan algoritma **sessizce geçersiz rota döndürür** ve kimse fark etmez — test stratejisindeki en kritik test sınıfı budur.

Sıcak başlangıç (warm start)

Kargo listesi her değiştiğinde SA **sıfırdan değil, önceki rotadan** başlar.

İki faydası vardır:

- **Hız** — iyi bir çözümden başlanır, daha az iterasyon gerekir
- **Kararlılık** — tek paket eklenince rota baştan aşağı değişmez; kullanıcı sistemin ne yaptığını anlar ve güvenir

Tohumlanmış rastgelelik + sıcak başlangıç birlikte "anında tetikleme"yi kullanılabilir kılar.

Neden savings ve 2-opt kullanılabiliyor

Çalışmanın erken aşamasında kapsam **zaman penceresi (VRPTW)** içeriyordu. O kapsamda bu iki yöntem **kullanılamazdı:**

- **Savings** tek depolu, penceresiz CVRP için tasarlanmıştır; birleştirme adımında pencere olurluluğunu değerlendiremez.
- **2-opt** bir rota segmentini **ters çevirir**. Ters çevirme segment boyunca tüm varış saatlerini kaydırır ve pencereleri ihlal eder.

Kapsam gün sınırına daraltılınca gerekçe ortadan kalktı:

simetrik matriste ters çevirme mesafeyi azaltıyorsa süreyi de azaltır, dolayısıyla süre kısıtı **basit bir toplam kontrolüyle** korunur.

Sonuç: literatürde en iyi dokümante edilmiş klasik kurulum kullanılabiliyor.

Problem sınıfı ve beklenti

CVRP **NP-zordur**. Durak sayısı arttıkça olası rota sayısı faktöriyel büyür; gerçek optimumu bulmak küçük örnekler dışında mümkün değildir.

Bu yüzden hedef "optimal çözüm" değil, ölçülebilir şekilde iyi çözümdür.

Kalite iki referansa karşı ölçülür:

- **Naive baseline** — insan dispatcher davranışı → tasarruf %
- **DFS-exhaustive** (n ≤ 10) — gerçek optimum → sapma %

İkincisi raporun en güçlü nicel iddiasını üretir:

"RouteWise, gerçek optimumun %X içinde."

Maliyet, rota hesaplandıktan SONRA eklenen bir rapor alanı değildir.

Amaç fonksiyonunun parçasıdır — algoritmanın her kararını bu değerler yönlendirir. "Sadece en kısa yol değil, değişkenler birlikte" cümlesinin kodda karşılığı burasıdır.

Amaç fonksiyonu

$$\min Z = \Sigma_r \left(\text{mesafe}(r) \cdot c_{km} + \text{yakıt}(r) \cdot c_{yakıt} + \text{süre}(r) \cdot c_{sürücü} \right) + \Sigma_i \text{öncelik}(i) \cdot p$$

r: rotalar · i: teslimatlar · c: birim maliyetler (cost_parameters) · p: öncelik ceza katsayısı

SERT KISITLAR

Aşılamaz. İhlal eden çözüm cezalandırılmaz, **reddedilir.**

Araç kapasitesi

Σ shipments.weight_kg ≤ vehicles.capacity_kg

araç tanımından gelir

Maksimum rota süresi

routes.total_duration_s ≤ max_route_duration_min × 60

"gün içinde bitsin" kuralı

AMAÇ FONKSİYONU BİLEŞENLERİ — minimize edilir

Mesafe maliyeti

toplam mesafe (km)

kaynak: yol grafı + Dijkstra

→ optimization_runs.total_distance_m

Yakıt maliyeti

(L/100km × km) × birim fiyat

tüketim araç bazlıdır

→ total_fuel_l · cost_parameters

Sürücü maliyeti

süre (saat) × saatlik ücret

servis süresi dahil

→ driver_cost_per_hour

Süre maliyeti

seyahat + servis süresi

yol grafındaki maxspeed'den

→ total_duration_s

Öncelik ihlali cezası

shipments.priority × katsayı

yumuşak kısıt — reddedilmez, pahalılaşır

Araç sabit maliyeti

fixed_cost_per_day

az araçla çözmeyi teşvik eder

opsiyonel bileşen

Parametreler nereden gelir

cost_parameters — DB · tarihli kayıt → her çağrıda → **Python servisi** — istekle alır

Yakıt fiyatı veya sürücü maliyeti değiştiğinde kod değişmez; optimizasyon sonuçları kendiliğinden değişir.

Neden "sadece en kısa yol" yetmez

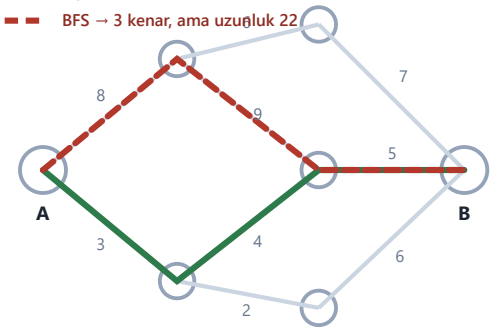
Aynı mesafedeki iki rota eşit değildir:

- Biri dar sokaklarda yavaş ilerler → **sürücü maliyeti** artar
- Diğeri daha uzun ama hızlı güzergâhtan gider → **süre ve yakıt** düşer
- Üçüncüsü yüksek öncelikli paketi güne çok geç bırakır → **ceza**

Yalnızca mesafeyi minimize eden algoritma bu üç farkı göremez.
RouteWise'ın amaç fonksiyonu görür.

SEVİYE 1 — YOL BULMA

"A noktasından B noktasına hangi yoldan gidilir?" · Bandırma yol grafı üzerinde · kayıt: path_comparison_runs



Dijkstra

Ağırlıklı grafta en kısa yolu **garanti eder**

REFERANS OPTİMUM

A*

Dijkstra ile **aynı yolu** daha az düğüm gezerek bulur

haversine sezgiseli

BFS

Kenar ağırlıklarını **YOK SAYAR**

en az kavşak ≠ en kısa

DFS

Optimallik garantisi yok

keşif sırasına bağlı

Rapor için değerli bulgu

Yol ağı **ağırlıklı** bir graftır; ağırlıkları yok sayan bir arama algoritması doğru cevabı veremez. BFS'in bu yanlışlığını sayısallaştırmak, algoritma seçiminin neden kritik olduğunu gösterir.

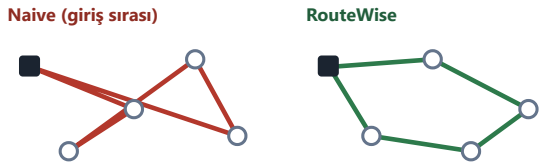
Ölçülen metrikler

yol uzunluğu (m) · hesaplama süresi (ms) · gezilen düğüm sayısı · Dijkstra optimumundan sapma (%)

Tümü `path_comparison_runs` tablosuna yazılır.

SEVİYE 2 — TUR SIRASI

"Hangi teslimat hangi sırayla, hangi araçla?" · mesafe matrisi üzerinde · kayıt: optimization_runs



Aynı duraklar, farklı sıra → farklı mesafe, süre, maliyet

Her koşuda naive baseline de hesaplanır ve kaydedilir: **tasarruf ölçümü bu karşılaştırmadan türetilir.**

Naive — giriş sırası

Taban çizgi: insan dispatcher davranışı. Optimizasyon yok.

Nearest Neighbor (greedy)

Açgözlü kurucu heuristik.
Çok hızlı, kalite orta.

Clarke-Wright savings

Klasik kurucu heuristik.
RouteWise'ın 1. adımı.

RouteWise — projenin kendi algoritması

savings (başlangıç) → 2-opt · relocate · exchange · Or-opt (yerel arama)
→ simulated annealing (tohumlanmış, sıcak başlangıçlı)

Her hamlede kapasite ve maksimum rota süresi olurluluğu kontrol edilir.

DFS-exhaustive

yalnızca n ≤ 10
Tüm permütasyonlar gezilir

GERÇEK OPTİMUM

Yöntem notu Hız karşılaştırması ancak implementasyonlar aynı dilde, benzer özenle, aynı veri yapıları üzerinde yazıldığında adildir. Her algoritma N kez koşturulur, medyan raporlanır.

Asıl ilginç sonuç tek başına hız değil, **kalite–süre dengesidir** — greedy hızlı ama kötü, DFS-exhaustive optimal ama kullanılamaz; RouteWise ikisinin arasındadır.

17 Analiz, Excel Raporları ve Tasarruf Yüzdesi

Projenin ana amacının ölçüldüğü yer

Tasarruf yüzdesi — ana metrik

$$\text{tasarruf \%} = (\text{cost}_{\text{naive}} - \text{cost}_{\text{routewise}}) / \text{cost}_{\text{naive}} \times 100$$

her ikisi de optimization_runs tablosunda, aynı plan_id altında

Her optimizasyon koşusunda naive baseline **de** hesaplanıp kaydedildiği için tasarruf geçmişe dönük agrege edilebilir.

Veri akışı — durumsuzluk korunur

Spring Boot — SQL agregasyon → **Python analytics** — pandas · scipy · matplotlib → **React** — Recharts · indirme

Excel raporu openpyxl

Ham sayılar — danışmanın kendi analizini yapabileceği tablo.

- **Sayfa 1 — Özet** — dönem · toplam km · yakıt L · maliyet TL · tasarruf %
- **Sayfa 2 — Koşu detayı** — her optimization_run satır satır · algoritma · metrikler
- **Sayfa 3 — Algoritma karşılaştırma** — Seviye 1 ve Seviye 2 tabloları · sapma % · süre ms
- **Sayfa 4 — Zaman serisi** — haftalık / aylık / yıllık kırılım · pandas resample
- **Sayfa 5 — İstatistiksel test** — Wilcoxon / t-test sonuçları · p değerleri · n

CSV dışa aktarma da aynı veriden türetilir.

PDF raporu matplotlib

Figürlerle birlikte sunuma ve teze hazır rapor.

- Kapak: dönem, depo, araç sayısı
- Tasarruf özeti — büyük punto, tek sayı
- Zaman serisi grafiği (haftalık/aylık/yıllık)
- Algoritma karşılaştırma tablosu
- Kalite–süre dengesi grafiği
- İstatistiksel anlamlılık sonucu
- Örnek rota haritası (önce / sonra)

Vektörel çıktı (SVG/PDF)
Tez raporuna doğrudan gömülür; ekran görüntüsü alınmaz, baskıda bozulmaz.

Görsel karşılaştırmalar

Grafik	Gösterdiği
Çubuk — algoritma × maliyet	hangisi ucuz
Çizgi — zaman × tasarruf %	iyileşme eğilimi
Dağılım — süre × sapma	kalite–hız dengesi
Yığılmış çubuk — maliyet kırılımı	yakıt / sürücü payı
Kutu grafiği — senaryo dağılımı	tutarlılık
Harita — önce / sonra rota	görsel kanıt

İki yerde, iki amaçla
React + Recharts → ekranda interaktif: hover, filtre
Python + matplotlib → raporda yayın kalitesi

İstatistiksel anlamlılık — iddiayı savunulabilir kılan adım

Tek başına "*RouteWise greedy'den %13 daha iyi*" zayıf bir iddiadır.
Jüri sorar: "Bu fark şans eseri olamaz mı? Kaç senaryoda denedin?"

Aynı senaryo kümesi tüm algoritmalarla koşturulur ve **eşleştirilmiş Wilcoxon işaretli sıra testi** uygulanır (dağılım varsayımı gerektirmez; alternatif paired t-test).

"RouteWise, greedy'ye göre anlamlı derecede daha iyi ($p < 0.001$, $n = 50$ senaryo)."

Maliyeti birkaç satır `scipy.stats` kodudur; kazancı, iddianın "benim sayılarım daha güzel"den savunulabilir bir bulguya dönüşmesidir. Analiz modülüne **baştan** konur.

Raporlanan metrikler

- Toplam mesafe (km)
- Yakıt tüketimi (L)
- Yakıt maliyeti (TL)
- Sürücü maliyeti (TL)
- **Toplam maliyet (TL)**
- Kullanılan araç sayısı
- Ortalama teslimat süresi
- ETA doğruluk sapması
- Hesaplama süresi (ms)
- **Tasarruf (%)**

Kırılımlar
Haftalık · aylık · yıllık — pandas resample ile optimization_runs.created_at üzerinden

Pazarlık payı olmayan kural

Sistem tek parça yazılmaz. Takvim sıkışırsa son çalışan katman sunulur; tek parça geliştirmede proje bitmezse ortada gösterilebilir hiçbir şey kalmaz.

1 KATMAN

Kargo listesi + adres doğrulama + tek araç rotası (TSP)

- Docker compose + PostGIS/pgRouting · Liquibase şema · seed filo
- OSM veri hattı: download → clip → load → export
- Kargo CRUD · haritadan konum seçimi · 5 katmanlı adres doğrulama · Dijkstra matris

Demo: "Paket ekliyorum, rota haritada yeniden çiziliyor."
Kapasite ve maliyet henüz yok. Tek araç, tek rota.
Risk: yol grafı kurulumu — en erken çözülmeli.

2 KATMAN

Çok araç + kapasite kısıtı + maliyet modeli (CVRP)

- Clarke-Wright savings → 2-opt / relocate / exchange / Or-opt → simulated annealing
- Amaç fonksiyonu: mesafe + yakıt + sürücü + öncelik · naive baseline kaydı
- **Olurluluk testleri** — kapasite ihlali asla üretilmemeli

Demo: "3 araçla dağıtım planı, TL cinsinden maliyet."
Naive planla farkı tablo halinde görünür.
Ölçüm altyapısı buradan itibaren hazır tutulur.

3 KATMAN

Maksimum rota süresi + ETA + bildirim + roller

- Çalışma günü süresi sert kısıt olarak algoritmaya girer
- Durak bazında ETA (maxspeed + servis süresi) · uygulama içi + e-posta bildirim
- JWT · üç rol · **rol sızıntısı testleri**

Demo: "Paketiniz tahmini 14:20'de teslim edilecek"
bildirimi alıcıya ulaşır; kurye kendi rotasını görür.

4 KATMAN

Dinamik yeniden rotalama

- Erteleme ("başka gün gelsin") · alıcı adres değişikliği · otomatik erteleme
- Teslim edilenler sabit, kalan duraklar yeniden optimize — son tamamlanan duraktan
- Beş güvenlik önlemi (sayfa 13) · sıcak başlangıç · debounce

Demo: Kurye yoldayken alıcı erteliyor,
rota canlı olarak yeniden hesaplanıyor.
Projenin "dinamik" iddiasının kanıtı.

5 KATMAN

Algoritma karşılaştırma modülü + istatistik raporları

- Seviye 1: Dijkstra · A* · BFS · DFS — yol grafı üzerinde
- Seviye 2: naive · greedy · savings · RouteWise · DFS-exhaustive
- pandas/scipy analiz · Excel + PDF dışa aktarma · haftalık/aylık/yıllık tasarruf

Projenin en görünür çıktısı
Ölçüm altyapısı Katman 2'den beri hazır olduğu için
bu katman veri üretmeyi beklemez.